ISPCBPlace: A Gradient-Based Placement Method for Irregular-Shaped PCBs under Complex Design Constraints

Lei Cai, Jixin Zhang*, Ke Cheng, Haiyun Li, Zhiwei Ye, and Mingyu Liu

***Abstract*—The placement of Printed Circuit Boards (PCBs) plays a critical role in modern electronic system design. In contrast to integrated circuit placement, which has been largely automated through advanced algorithms, PCB placement still relies heavily on manual efforts. This is primarily due to the restrictive constraints that devices with vastly different sizes and irregular shapes must be placed within extremely high-density, high-granularity, and limited spaces in the placement process of modern PCBs. Moreover, routing constraints such as the routability of surface-layer traces must be taken into account. These challenges have significantly hindered the advancement of fully automated industrial PCB placement algorithms for many years. To address the challenges, we propose ISPCBPlace, a gradient-based placement method for modern irregular-shaped PCBs under complex design constraints. We formulate the PCB global placement problem as a gradient-based optimization task and propose three key gradient formulation, boundary gradient, overlap gradient (incorporating a centrifugal gradient), and half-perimeter wire-length gradient, to enable effective convergence while satisfying design constraints. Furthermore, we propose a module-aware clustering method to enhance routability of surface-layer traces, and introduce a simulated annealing-based fine-tuning strategy to optimize component orientations and ensure legalization. Experimental results within real-world industrial PCB cases demonstrate that ISPCBPlace outperforms the state-of-the-art methods and even manual designs in terms of wire length, and successfully meeting industrial constraints.**

***Index Terms*—High-Density, Irregular-Shaped, Gradient Optimization, Printed Circuit Board Placement.**

I. Introduction

IRREGULAR-SHAPED PCBs are about to become a key ingredient in modern, compact, innovative product design. These provide enormous benefits that range from optimized space and flexibility in design to higher functionality for a variety of uses in wearables, medical devices, and automotive systems.

The placement of irregular-shaped PCBs plays a crucial role in the PCB design process. However, it has largely relied on manual design due to the numerous placement constraints inherent in PCB, as shown in Fig. 1. PCB placement automation has long been constrained by a set of challenges such as irregular board geometries, non-uniform component sizes, stringent routability requirements, and complex design rules that resist differentiable formulation.

Currently, PCB placement automation remains largely reliant on traditional algorithms, such as heuristic and stochastic methods [1]–[6], and the development of automated PCB placement continues to face significant challenges and difficulties.

Although numerous approaches have been introduced for integrated circuit (IC) placement, including AI-driven [7]–[12] and gradient-based methodologies [13]–[19], the problems they address differ significantly from those in PCB placement. IC placement methods have not encountered the same stringent challenges and complex constraints that are characteristics of PCB layout. Unlike IC placement, where standardized cells are optimized through approximate half-perimeter wire-length (HPWL) and density, PCB designs typically involve heterogeneous components in PCB layouts featuring irregular and highly constrained placement spaces, which requires extremely high-granularity optimization. Moreover, key objectives such as routability can only be evaluated after routing, resulting in long feedback loops that hinder the efficient exploration of placement candidates. As shown in Table I, IC placement methods cannot be directly applied to PCB placement due to the significant challenges inherent in PCB design.

To address the above challenges, we aim to propose a high-efficiency gradient-based placement framework tailored to real-world PCB placement. Our solution focuses on transforming irregular PCB placement optimization into a gradient-based optimization process while meeting complex PCB constraints. However, conventional gradient-based methods are not directly applicable due to the presence of irregularly shaped components of varying sizes and non-rectangular PCB boundaries.

To overcome the above limitations, we propose ISPCBPlace, a gradient-based placement framework designed for modern irregular-shaped PCBs with complex design constraints. Specifically, we formulate the PCB global placement task as a gradient-based optimization problem and derive three key gradient formulations, boundary gradient, overlap gradient (incorporating with a centrifugal gradient), and HPWL gradient, to enable effective convergence while adhering to design rules. Additionally, we develop a module-aware clustering strategy to improve the routability of surface-layer traces and introduce a simulated annealing (SA)-based fine-tuning strategy to optimize device orientations, ensuring that the final layout is both legal and valid.

The main contributions are sumarized as follows:

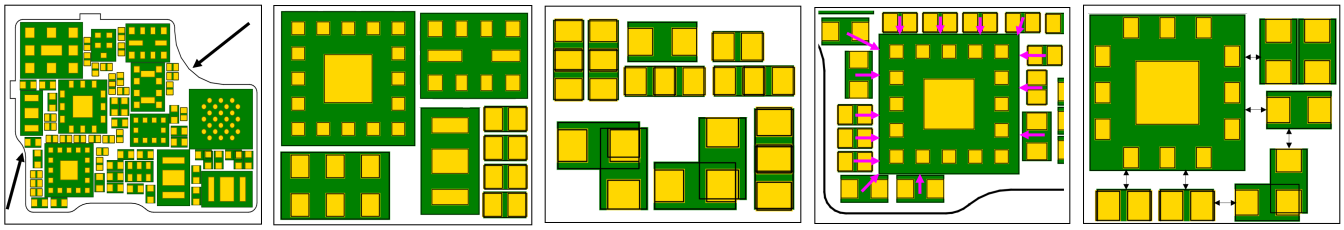
This work is partially supported by the National Natural Science Foundation of China under Grant No. 62441233, 62002106, and Huawei Device Co., Ltd.

Lei Cai, Jixin Zhang, Ke Cheng, and Zhiwei Ye are with the School of Computer Science, Hubei University of Technology, Wuhan 430068, China. (E-mail: zhangjx@hbut.edu.cn)

Haiyun Li is with the Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China

Mingyu Liu is with the Wuhan Research Institute, Huawei Device Co., Ltd., Wuhan 430200, China

*Corresponding author: Jixin Zhang (zhangjx@hbut.edu.cn)



(a) Irregular boundary (b) Differently sized devices (c) Compatible devices (d) Surface-layer traces (e) Spacing

Fig. 1. Characteristics of Modern Irregular PCB Layout.

TABLE I
COMPARISON OF PLACEMENT CHARACTERISTICS BETWEEN INTEGRATED CIRCUITS AND MODERN IRREGULAR-SHAPED PCBS

Category	Integrated Circuits	Modern Irregular-Shaped PCBs
Placement Region	Relatively spacious	Extremely limited and high-density
Boundary Shape	Regular (rectangular)	Highly irregular and non-rectangular
Placement Components	Standardized cells and regular macro blocks	Heterogeneous devices with large size variations and irregular shapes
Objective	Approximate Wirelength + Density	Wirelength + Routability of Surface-Layer Traces
Methods	Automated (AI-based [7]–[12], analytical [13]–[19])	Predominantly manual, heavily dependent on experts' experience
Constraints	Overlap + Region + Alignment	Overlap + Irregular Boundary + Spacing

- To our best knowledge, we are first in the literature to propose gradient-based optimization approach for modern irregular-shaped PCB placement, addressing complex PCB constraints.
- We formulate PCB placement problem as a gradient-based optimization problem, and propose three key gradient components, boundary gradient, overlap gradient, and HPWL gradient, to simultaneously achieve placement objectives and satisfy design constraints.
- To manage significant disparities in device sizes during global placement, we model overlap penalties using a centrifugal gradient formulation, enabling more effective convergence.
- To handle irregular PCB boundaries, we compute the geometric complement between the irregular boundary and its enclosing rectangle, and derive gradients based on overlaps between devices and this complement region.
- We develop a module-aware clustering approach that integrates surface-layer trace constraint into the placement process, enhancing routability of surface-layer traces.
- The experimental results within real-world industrial PCB cases demonstrate that ISPCBPlace outperforms the state-of-the-art methods and even manual designs in terms of wire length, and successfully meeting industrial constraints.

The remainder of this article is organized as follows. Section II describes the background and related works. Section III formally defines the research problem. Section IV explains the detailed implementation. Section V demonstrates the results. Section VI concludes this article.

II. RELATED WORKS

A. Placement Method

1) Heuristic Placement Method

Heuristic methods have long been used to address the NP-hard placement problem. Representative approaches include simulated annealing with sequence-pair representations [20] and genetic algorithms for cell placement [21], which have been further extended to domains such as thermal-aware 3D IC placement [22], FPGA task scheduling [23], analog layout generation [24], and yield-driven analog optimization [25]. Despite their versatility, these heuristic-based methods generally suffer from limited efficiency and poor scalability, making them inadequate for large-scale PCB placement under complex constraints.

2) Analytical Placement Method

Analytical placement techniques have demonstrated strong scalability and efficiency. Representative works include electrostatics-based placement for large-scale mixed-size designs [13], refined density modeling for local overflow control [14], and GPU-accelerated gradient-based frameworks that reformulate placement as neural network training [15]. Nevertheless, these methods are not directly applicable to PCB placement, as they do not adequately address irregular board outlines, heterogeneous component geometries, complex constraints, or surface-layer routability.

3) AI-based Placement Method

Deep learning has recently advanced IC placement, with representative works including RL-based graph placement [7], visual representation learning in MaskPlace [8], and transformer-based transferable placement in ChipFormer [9]. For analog ICs, prior studies have examined both the limitations of analytical placement [18] and deep learning-based solutions such as DeepPlacer [12]. However, despite their promising results in IC domains, these AI-based methods still struggle to accommodate the diverse and complex constraints of modern PCB placement.

B. PCB Placement Method

1) Traditional PCB Placement Method

Traditional PCB placement has predominantly relied on heuristic and evolutionary optimization techniques. Early studies employed genetic algorithms (GAs) to automate component placement [1], with subsequent improvements such as

Self-Organizing GAs to enhance convergence and solution quality [2]. Hybrid approaches further combined GAs with routing algorithms to jointly optimize placement and routing [3]. Beyond GAs, particle swarm optimization has been explored for multi-objective PCB placement [4], while simulated annealing has been adopted in recent tools such as SA-PCB to enable stochastic search for PCB layouts [5]. Despite their effectiveness on small-scale to medium-scale problems, these heuristic-based methods generally exhibit limited scalability and optimization efficiency. As a result, they struggle to handle large-scale PCB layouts under complex design constraints.

2) Recent PCB Placement Method

Recent PCB placement research has explored AI-based and analytical approaches. Reinforcement learning has been explored for PCB placement with adaptive reward design [6], but remains limited to regular-shaped boards and small-scale problems, and fails to fully model complex PCB constraints. Analytical PCB placement, such as net separation-oriented optimization [26], adopts ePlace/RePlace-like objectives [13], [14] and lacks sufficient expressiveness for highly constrained and irregular PCB layouts. Reference-based placement has been proposed to exploit layout reuse via subgraph matching [27], but its effectiveness depends heavily on the availability of similar designs, while diversity-aware extensions still struggle with highly irregular and heterogeneous layouts [28]. Overall, despite these advances, existing PCB placement techniques remain inadequate for industrial-scale scenarios involving irregular board outlines, heterogeneous device geometries, stringent constraints, and large-scale designs.

III. Problem Formulation

A. Definition

Given a $Netlist$ which consists of nets $\mathcal{N}$ and n devices $\mathcal{D}$, a set of pins $\mathcal{P}$, a set of constraints $\mathcal{C}$, perform a gradient-based placement process for each device $D_i \in \mathcal{D}$, incorporating constraints $C_j \in \mathcal{C}$ into the optimization objective $\mathcal{L}$, such that constraint violations are minimized throughout the optimization trajectory and ultimately eliminated. The detailed notation and key terms are shown in Table II.

TABLE II
Notation and Key Terms

Symbol	Description
$\mathcal{D} = \{D_1, D_2, \ldots, D_n\}$	Set of devices
$\mathcal{N} = \{N_1, N_2, \ldots, N_k\}$	Set of nets
$\mathcal{P} = \{p_1, p_2, \ldots, p_m\}$	Set of pins
$\mathcal{MG} = \{G_1, G_2, \ldots, G_l\}$	Set of module groups
$\mathcal{B} \subset \mathbb{R}^2$	Irregular PCB boundary
$\mathcal{C} = \{C_1, C_2, \ldots, C_n\}$	Set of PCB constraints
$\mathcal{S} = \{(x_i, y_i, w_i, l_i, \theta_i)\}_{i=1}^{n}$	PCB Placement State
∇	Gradient differentiation

B. Objective

In ISPCBPlace, the gradient objective is to minimize the optimization function $\mathcal{L}$ defined in:

$$\min \ \mathcal{L} = \text{Wirelength} + \text{Routability}_{SLT}, \quad \text{via } \nabla\mathcal{L} \text{ and s.t. } \mathcal{C} \tag{1}$$

where Wirelength represents the sum of the HPWL calculated for the corresponding nets of the components, and Routability_{SLT} is the routability of surface-layer traces (SLT). The objective for routability of surface-layer traces is to facilitate efficient surface-layer trace routing under limited routing resources. $\mathcal{C}$ is the constraints for irregular PCB placement. The details of $\mathcal{C}$ are shown in following.

C. Constraints

The placement of devices $\mathcal{D}$ must satisfy the following constraints $\mathcal{C}$:

- C_1: **Spacing Constraint**. A minimum spacing distance must be set for each pair of devices:

$$|D_i - D_j| \geq d_{\text{min}} \quad \forall i \neq j \tag{2}$$

  where $|\cdot|$ denotes the Euclidean distance.
- C_2: **Compatible Pin Grouping Constraint**. Pins of devices within a compatible group must be placed adjacently to ensure signal integrity. Let C be a compatibility matrix where $C(i,j) = 1$ indicates that devices D_i and D_j are compatible. The constraint is:

$$|D_i - D_j| \leq d_{\text{comp}}, \quad \forall\, C(i,j) = 1, \tag{3}$$

  where d_{comp} is the maximum allowed separation for compatible pairs.
- C_3: **Overlap Constraint**. Ensures that no devices overlap in the layout. All device pairs must satisfy:

$$\text{Overlap}(D_i, D_j) = 0 \quad \forall i \neq j \tag{4}$$

  where $\text{Overlap}(\cdot,\cdot)$ computes the overlapping area between devices D_i and D_j.
- C_4: **Boundary Constraint**. Ensures that all devices are properly placed within the irregular and limited space of the design boundary:

$$D_i \in \mathcal{B} \quad \forall i \tag{5}$$

  where $\mathcal{B}$ represents the irregular-shaped PCB boundary.

D. Gradient formulation

To optimize the placement objective $\mathcal{L}$ while satisfying the set of constraints $\mathcal{C}$, we reformulate the PCB placement task as a gradient-based optimization problem. Considering the wirelength objective together with the overlap and boundary constraints, the optimization can be expressed in terms of the following gradient formulation:

- **Gradient 1: Wirelength Gradient** This gradient pulls connected devices closer by minimizing the half-perimeter wirelength (HPWL). It is computed based on the bounding box of pins for each net.

$$\nabla Wirelength = \nabla \sum_{i}^{|\mathcal{N}|} \text{HPWL}(N_i) \tag{6}$$

- **Gradient 2: Overlap Gradient** (C_3) For devices with heterogeneous sizes, we define the overlap gradient as

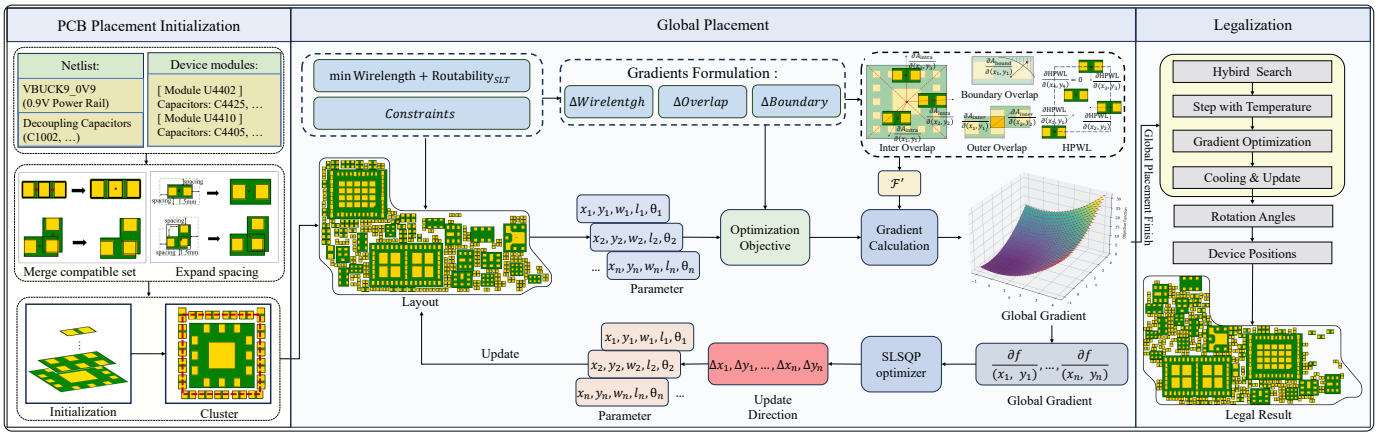


Fig. 2. Framework of ISPCBPlace. The process begins with layout initialization, followed by the preprocessing of PCB constraints and formulation of the placement objective. The gradient-based optimization for PCB placement is then defined, with the HPWL gradient, overlap gradient (incorporating a centrifugal gradient), and boundary gradient formally modeled to guide the global optimization process. Finally, a simulated annealing-based fine-tuning strategy is applied to optimize component orientations and ensure legalization.

the sum of the inter-device overlap gradient and the intra-device overlap gradient, formulated as:

$$\nabla Overlap = \nabla A_{\text{inter}} + \nabla A_{\text{intra}}, \tag{7}$$

where ∇A_{inter} represents the overlap between distinct devices, while ∇A_{intra} captures the overlap within a single device, particularly when a smaller device is fully enclosed by a larger one.

- **Gradient 3: Boundary Gradient** (C_4) This term penalizes devices that extend beyond the irregular PCB boundary. Specifically, we measure the overlap area between each device and the exterior region of the boundary, obtained via a bounding-box complement operation, and compute its gradient. The formulation is given by:

$$\nabla Boundary = \nabla A_{\text{bound}}, \tag{8}$$

where A_{bound} captures the directional gradients of the overlap area with respect to each device's position.

Furthermore, to enhance the Routability$_{SLT}$, we propose some mechanisms, such as a module-aware clustering method, and approximate pairwise penalty modeling. Besides, the **Spacing Constraint** (C_1) and the **Compatible Pin Grouping Constraint** (C_2) are pre-processed prior to optimization, ensuring that the placement consistently satisfies these requirements throughout the placement process. Since it is difficult to drive the objective gradients to zero, a legal placement stage is performed after global placement to guarantee that the final layout strictly adheres to all constraints.

IV. Methodology

A. Framework of ISPCBPlace

In this work, we propose ISPCBPlace, a comprehensive gradient-based optimization framework for irregular PCB placement considering complex constraints. As shown in Fig. 2, ISPCBPlace begins by mathematically modeling the PCB layout, including device geometries and design constraints, to facilitate gradient-based optimization. The layout is then initialized using a module-aware clustering approach to enhance Routability$_{SLT}$. Subsequently, ISPCBPlace gradientizes HPWL by analyzing the maximum bounding box of each net. Besides, ISPCBPlace computes overlap gradient incorprating a centrifugal gratient, and irregular PCB boundary gradient. ISPCBPlace employs a gradient-based optimizer for nonlinear constrained problems. During each iteration, ISPCBPlace balances the weights of wirelength and overlap gradients, ensuring that wirelength optimization is not overly influenced by overlap reduction. Finally, a fint-tuning strategy combining simulated annealing for discrete angle adjustments with gradient-based refinement for continuous parameters ensures that the final placement adheres to all hard design constraints.

B. PCB Placement Initialization

Before optimization begins, the PCB placement undergoes a series of preprocessing steps. This includes formulating device geometries, parsing netlists, and extracting constraint information. Based on the processed inputs, we initialize device positions and orientations to provide a feasible starting point for subsequent placement optimization.

1) Geometric representation of devices

In the gradient-based optimization, each device i is represented by its center position (x_i, y_i), size (w_i, l_i), and rotation angle θ_i. The placement state for all n devices is

$$\mathcal{S} = [s_1, s_2, \ldots, s_n]^\top, \qquad s_i = [x_i,\ y_i,\ w_i,\ l_i,\ \theta_i]^\top. \tag{9}$$

Here the primary optimization variables are the center coordinates (x_i, y_i) and device dimensions w_i, l_i. Rotation θ_i is restricted to the discrete set $\{0°, 90°, 180°, 270°\}$.

2) Constraint Preprocessing

We consider incorporating constraints related to devices through preprocessing prior to initialization. The details are as follows.

- **Spacing Constraint Between Devices.** (C_1) We transform spacing constraint to overlap constraint by expanding device boundaries. As shown in the Fig. 2, the devices are respectively expanded with a spacing of 1.5 mm.

- **Compatibility Constraint Between Devices.** (C_2) Devices within a compatible group are fused into a composite unit according to predefined rules that determine their relative geometry, including relative offsets, orientations, and pin alignments.

3) Module-Aware Clustering

To enhance the surface-layer routability, we propose a module-aware clustering strategy before global placement, as shown in Algorithm 1. Initially, all devices are partitioned into distinct module groups $\mathcal{MG}$ according to predefined design rules. For each group, it is classified as center chip c_0, critical module $\mathcal{G}_c$, non-critical module $\mathcal{G}_n$, and other module $\mathcal{G}_o$. The center chip c_0 is first placed at the geometric center of the module. Subsequently, for each submodule type $\mathcal{G} \in \mathcal{G}_c, \mathcal{G}_n, \mathcal{G}_o$, a reference surface $\mathbf{P}$ is constructed as a convex hull based on the hierarchy of $\mathcal{G}$. For each device D belonging to the current submodule, its placement is constrained to lie within the N-ring neighborhood of the reference surface, denoted by $D \in \partial^N \mathbf{P}$. Devices $D \in \partial^N \mathbf{P}$ are constrained to be placed within this region as much as possible. And any devices placed outside the region will receive a penalty. Within the constrained region, device positions are refined using Algorithm 2. The Algorithm 2 is applied to the current module cluster under the corresponding surface-layer constraints, rather than to the entire design. After convergence at the current stage, the optimized device positions are fixed to form a cluster. Once all clusters have been processed, a legalization procedure is applied to each group using Algorithm 3. The resulting clusters are then passed to the global placement stage.

Algorithm 1 Module-Aware Clustering

Require: Device set $\mathcal{D}$, ring index N
Ensure: Clustered placement $\mathcal{S}_{\text{init}}$

```
Partition D into several module groups {MG} according to predefined design rules
for all G ∈ MG do
    Decompose G into center chip c_0, critical module G_c, non-critical module G_n, and other module G_o
    Place c_0 at center of G: c_0 ← (G_x, G_y)
    for all G ∈ {G_c, G_n, G_o} do
        Construct reference surface P:
            P = { c_0,                          G = G_c,
                { Hull({D | D ∈ G_c ∪ c_0}),    G = G_n,
                { Hull({D | D ∈ G_n ∪ G_c ∪ c_0}), G = G_o,
        for all D ∈ G do
            Constrain D to lie within the N-ring neighborhood of P: D ∈ ∂^N P
            Add penalty: Penalty = α||D − P||_2, D ∉ ∂^N P
            Refine placement of D via the GRADIENT-BASED OPTIMIZATION PROCESS (Algorithm 2):
            D ← arg min_D (F(D; P) − Penalty)
        end for
        Fix positions of all D ∈ G to form a cluster
    end for
end for
for all G ⊆ {MG} do
    S_k ← Legalize(G), using Algorithm 3 within groups
end for
return S_init ← ⋃_k S_k
```

C. Global Placement

In this part, we formulate different objective and constraints into three gradients (wirelength gradient, overlap gradient (C_3) and boundary gradient (C_4)) in PCB placement process. For brevity, we only discuss the equations in the x dimension, as those in the y dimension are similar.

1) Wirelength Gradient

Different from [13]–[15], which adopt a smooth approximation of HPWL, we employ a more high-granularity HPWL calculation method tailored for PCB placement due to the high granularity requirements and extremely limited available space in PCB layouts, where direct differentiation of HPWL is inherently challenging. Specifically, we propose a method of HPWL gradient calculation by analyzing HPWL boundaries, where devices located on the HPWL boundary possess a gradient, while devices positioned within the boundary exhibit a gradient of zero. For the PCB layout, the HPWL is defined over all nets $\mathcal{N}$ as:

$$\mathrm{HPWL} = \sum_{n\in\mathcal{N}} \Big((x_{\max}(n) - x_{\min}(n)) + (y_{\max}(n) - y_{\min}(n)) \Big), \tag{10}$$

where $x_{\max}(n) = \max_{p\in P_n} x_p$ and $x_{\min}(n) = \min_{p\in P_n} x_p$ are the extreme x-coordinates of pins in net n, and analogously for y. And the P_n represents the pins in the net n. Then, we define the sets of extreme pins for net n:

$$A_x^+(n) = \arg\max_{q\in P_n} x_q, \qquad A_x^-(n) = \arg\min_{q\in P_n} x_q, \tag{11}$$

where $A_x^+(n)$ and $A_x^-(n)$ represent the pins with the largest x-coordinates and the smallest x-coordinates, respectively. The gradient with respect to pin coordinates is given by:

$$\frac{\partial\,\mathrm{HPWL}}{\partial x_p} = \sum_{n:p\in P_n} \big(\alpha_p^{x,+}(n) - \alpha_p^{x,-}(n)\big), \tag{12}$$

where the coefficients satisfy:

$$\alpha_p^{x,+}(n) \geq 0, \qquad \sum_{q\in A_x^+(n)} \alpha_q^{x,+}(n) = 1, \tag{13}$$

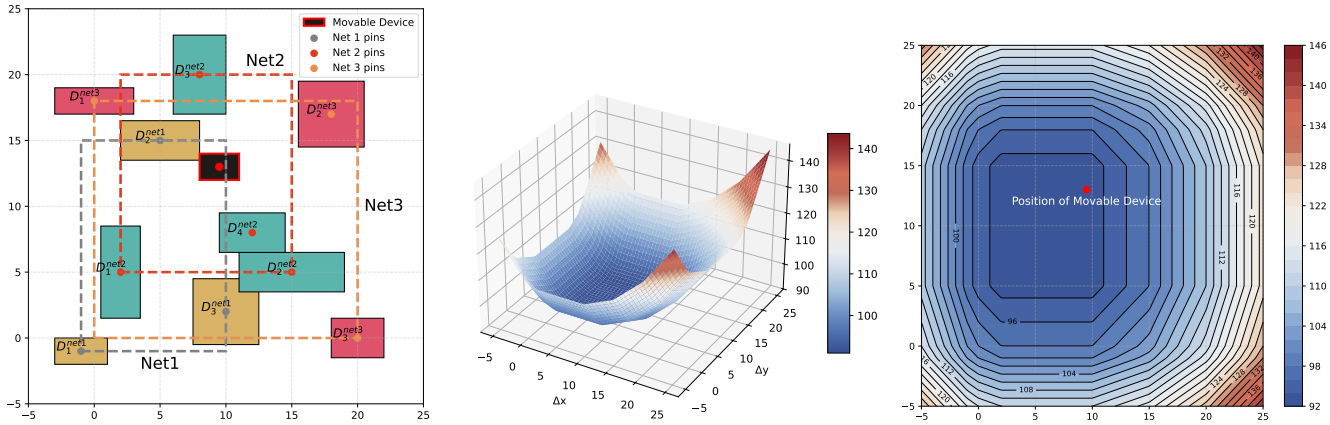
$$\alpha_p^{x,+}(n) = 0 \quad (p \notin A_x^+(n)). \tag{14}$$

where $\alpha_p^{x,+}(n)$ represents the HPWL gradient contributions for the pins in the set $A_x^+(n)$. If multiple pins belong to $A_x^+(n)$, the coefficients are evenly distributed. It is worth noting that pins located within the HPWL boundary have an HPWL gradient of zero. And the definition of $\alpha_p^{x,-}(n)$ is similar. For device i with pins D_i^{pins}, each pin position can be expressed as:

$$x_p = x_i + \Delta x_{ip}, \tag{15}$$

where (x_i, y_i) is the device center and $(\Delta x_{ip}, \Delta y_{ip})$ is the pin offset. Thus, the device-level gradient becomes

$$\frac{\partial \mathrm{HPWL}}{\partial x_i} = \sum_{p\in D_i^{pins}} \sum_{n:p\in P_n} \big(\alpha_p^{x,+}(n) - \alpha_p^{x,-}(n)\big). \tag{16}$$



(a) 2D Layout for nets (b) HPWL vs. Device Position (c) HPWL Contour Plot

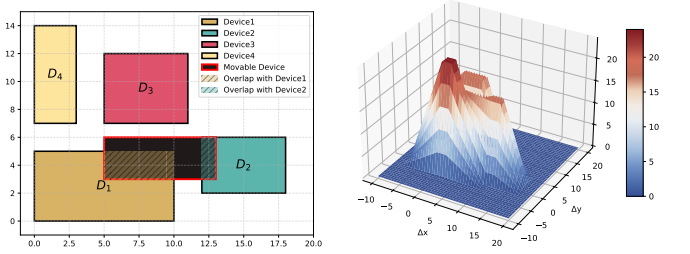
Fig. 3. (a) A 2D layout view showing the movable device (red), connected to pins of three nets. Dashed lines indicate the connections from the movable device to each net's boundary. (b) A 3D surface plot of HPWL as a function of device displacement $(\Delta x, \Delta y)$: lower HPWL values appear near the surface valley, rising sharply toward the edges. (c) A contour plot of the same HPWL landscape in the $(\Delta x, \Delta y)$ plane. The red dot marks the movable device.

Only boundary elements of net affect the HPWL derivative, and interior elements do not influence gradient-based updates unless they reach extremal status, as shown in Fig. 3. Although HPWL is piecewise-constant almost everywhere, the HPWL gradient calculation correctly expose this structure by zero gradients off extremal paths. And the gradient of wirelength is represented as:

$$\nabla_x \mathrm{WL}(\mathcal{S}) = \left(\frac{\partial \mathrm{HPWL}}{\partial x_i}, ..., \frac{\partial \mathrm{HPWL}}{\partial x_n} \right)^\top. \quad (17)$$

2) Overlap Gradient (C_3)

In this part, we define the device overlap gradient. When two devices intersect, their overlap is calculated as:



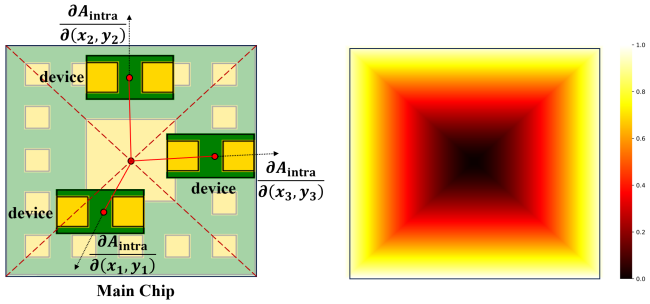
(a) 2D View of devices (b) Total Overlap vs Relative Position

Fig. 4. (a) Top-down 2D view of overlapping devices, and a movable Device (red outline). (b) A 3D surface plot showing total overlap area as a function of relative displacement $(\Delta x, \Delta y)$ of the movable device.

$$A_{\mathrm{inter}}(i; x_i, y_i) = \sum_{j \neq i}^{n} \mathrm{Area}\big(D_i(x_i, y_i) \cap D_j(x_j, y_j)\big), \quad (18)$$

where D_i and D_j represent the positions of the devices. We approximate the gradient of the overlap area $A_{\mathrm{inter}}(i)$ using central finite differences:

$$\frac{\partial A_{\mathrm{inter}}(i)}{\partial x_i} = \frac{A_{\mathrm{inter}}(i; x_i + \delta, y_i) - A_{\mathrm{inter}}(i; x_i - \delta, y_i)}{2\delta}, \quad (19)$$



(a) Escape from chip area (b) Escape heatmap

Fig. 5. Schematic diagram of centrifugal gradient. (a) All devices within the main chip align with the centrifugal gradient direction, which guides them outward to escape from the chip interior. (b) Schematic illustration of the centrifugal gradient field within the main chip, visualized as a repulsive force field where darker regions correspond to higher force intensity.

where δ is a small perturbation step size. As shown in Fig. 4, overlap gradients naturally arise between devices. Besides, this formulation generalizes to irregularly shaped devices. The inter-device overlap calculation is particularly suitable for PCB placement, where high granularity, high density, and limited spatial resources are critical.

In order to address the issue of vanishing overlap gradients when small devices are placed inside larger ones, we propose a centrifugal gradient. The idea is to impose a repulsive effect that pushes the smaller device away from the center of the larger device, thereby ensuring a continuous and directional gradient. As illustrated in Fig. 5, for device i, the internal overlap area A_{intra} is defined as the overlap area between different parts within the boundary of the larger device:

$$A_{\mathrm{intra}}(i; x_i, y_i) = (1 - \frac{r}{r_0}) * Overlap(i, j), \quad (20)$$

$$r = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}, \quad (21)$$

where r denotes the distance from the center of device i to the boundary of the larger device, and r_0 is the maximum escape distance within the larger device. The ratio r/r_0 plays a

critical role in ensuring continuity of the overall gradient field. Specifically, the centrifugal gradient is active only when the smaller device is strictly inside the larger one ($r < r_0$), and its magnitude smoothly decays to zero as the device approaches the boundary. When the smaller device reaches the boundary of the larger device ($r = r_0$), the centrifugal term becomes exactly zero, and the gradient contribution is seamlessly taken over by the standard inter-device overlap gradient. Similarly, the centrifugal gradient is represented as:

$$\frac{\partial A_{\text{intra}}(i)}{\partial x_i} = \frac{A_{\text{intra}}(i; x_i + \delta, y_i) - A_{\text{intra}}(i; x_i - \delta, y_i)}{2\delta}. \tag{22}$$

The inter-device overlap corresponds to the overlap between different devices, while the intra-device overlap pertains to the overlap within a single device. The total device overlap gradient is thus expressed as:

$$\nabla_{x_i} Overlap(\mathcal{S}) = \frac{\partial A_{\text{inter}}(i)}{\partial x_i} + \frac{\partial A_{\text{intra}}(i)}{\partial x_i}. \tag{23}$$

3) Boundary Gradient (C_4)

Due to the difficulty of directly defining the irregular boundaries of the board in horizontal and vertical coordinates for gradient optimization, we designed a unique boundary gradient to handle irregular boundaries. In order to ensure that all devices are located inside the irregular PCB, we first constrain them in the boundary of circumscribed rectangle:

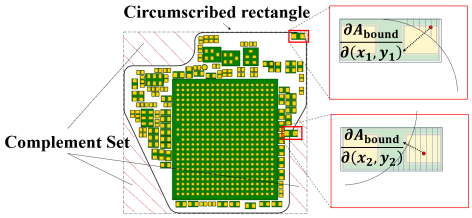


Fig. 6. The complement set refers to the area between the irregular PCB boundary and its circumscribed rectangle, as illustrated by the region filled with red dashed lines.

$$x_{\min} \le x_i \le x_{\max}, \quad y_{\min} \le y_i \le y_{\max}, \tag{24}$$

where $(x_{\min}, y_{\min})$ and $(x_{\max}, y_{\max})$ are the coordinates of the outer rectangle boundary. Then we design the complement set, the area between the boundary and its circumscribed rectangle, and incorporate it into the boundary gradient calculation. The complement set between the shielding cover boundary and the outer rectangle is given by:

$$B^{\text{out}}_{\text{boundary}} = \left(\text{BBox}(\mathcal{B}) \setminus \mathcal{B}\right), \tag{25}$$

$$A_{\text{bound}}\left(i; x_i, y_i\right) = \text{Area}\left(D_i(x_i, y_i) \cap B^{\text{out}}_{\text{boundary}}\right), \tag{26}$$

where $\mathcal{B}$ represents the boundary of the irregular PCB, $\text{BBox}(\mathcal{B})$ represents the outer rectangle boundary, and A_{bound} is the overlap that the device is incorporated into the complement set. As shown in Fig. 6, the coordinate space for gradient optimization is confined within the bounding rectangle. We approximate the gradient of the boundary-overlap term with respect to a device's position (x_i, y_i) using central finite differences:

$$\frac{\partial A_{\text{bound}}(i)}{\partial x_i} = \frac{A_{\text{bound}}(i; x_i + \delta, y_i) - A_{\text{bound}}(i; x_i - \delta, y_i)}{2\delta}, \tag{27}$$

where δ is a small perturbation step. And the gradient of boundary is calculated by:

$$\nabla_x Boundary(\mathcal{S}) = \left(\frac{\partial A_{\text{bound}}}{\partial x_1}, ..., \frac{\partial A_{\text{bound}}}{\partial x_n}\right)^\top. \tag{28}$$

As the gradients with respect to device overlap and boundary violation are both related to overlap handling, we combine them into a single formulation:

$$\nabla_x \text{O}(\mathcal{S}) = \nabla_x Overlap(\mathcal{S}) + \gamma \nabla_x Boundary(\mathcal{S}), \tag{29}$$

where γ denotes the weighting factor for the boundary gradient, and keeping devices within the boundary is much more critical than minimizing device overlap, therefore it is assigned a large value of 10.

4) Gradient-Based Optimization Process

To improve placement quality under multiple constraints, we formulate the overall optimization gradient $\nabla\mathcal{F}$ as a weighted sum of individual gradient components:

$$\nabla\mathcal{F}(\mathcal{S}) = \nabla \text{WL}(\mathcal{S}) + \lambda \cdot \nabla \text{O}(\mathcal{S}) \tag{30}$$

The composite gradient $\nabla\mathcal{F}$ provides the unified optimization direction for updating device coordinates at each iteration during the global placement stage.

Algorithm 2 Gradient-Based Optimization Process

Require: Initial solution $\mathcal{S}_0$, objective function $\mathcal{F}(\mathcal{S})$, constraint function $\text{C}_{\text{viol}}(\mathcal{S})$, maximum iterations N
Ensure: Optimal solution $\mathcal{S}^*$

```
Initialize S ← S_0, Lagrange multiplier λ_0, step size α_max, tolerance ε
for t = 0 to N − 1 do
    Evaluate ∇F(S), C_viol(S), and ∇C(S)
    if ‖∇F(S)‖ < ε and C_viol(S) ≤ ε then
        return S* ← S {Converged solution}
    end if
    Solve QP subproblem for update direction ΔS:
        min_ΔS   ∇F^T ΔS + (1/2) ΔS^T H ΔS
        s.t.     C_viol(S + ΔS) = 0
    Update S ← S + α · ΔS, with α ∈ (0, α_max]
    Update λ ← λ + ∇C_viol · ΔS
    Optionally adjust step size: α ← decay(α)
end for
return S* ← S
```

During global placement, we employ a continuation-based weighting strategy to balance the inherently non-convex overlap objective and the wirelength optimization. Specifically, we gradually increase the overlap penalty weight $\lambda(t)$ over iterations, allowing the optimizer to transition smoothly from HPWL-driven placement to overlap-aware refinement. To preserve the quality of wirelength optimization and mitigate the

risk of being trapped in poor local minima, $\lambda(t)$ is scheduled to grow slowly:

$$\lambda^{(t)} = \max(\frac{1}{10} \cdot t, 3). \tag{31}$$

At the early stage ($t = 0$), the overlap weight is negligible, and the optimization is dominated by HPWL minimization, which provides a relatively smooth and well-behaved objective landscape. As t increases, $\lambda(t)$ is gradually enlarged, and overlap penalties are introduced progressively. Importantly, during this process, the descent directions induced by overlap gradients remain constrained by the already optimized HPWL structure. This gradual reweighting scheme effectively avoids directly optimizing a strongly non-convex overlap objective from scratch. Instead, overlap elimination is performed as a smooth deformation of an HPWL-optimized placement, enabling the optimizer to continuously reduce overlaps while preserving global wirelength quality.

To solve the constrained placement problem, we introduce the Sequential Least Squares Quadratic Programming (SLSQP) algorithm [29], a standard gradient-based optimizer for nonlinear constrained problems. Since Routability_{SLT} is difficult to formulate explicitly as a differentiable objective function, we model it indirectly by expressing routing-related requirements in the form of SLSQP-compatible constraint functions $\text{C}_{\text{viol}}(\mathcal{S})$:

$$\begin{aligned}\text{C}_{\text{viol}}(D_i, D_j) = {} & w_1 \cdot \max(1 - \mathbf{1}_{\text{face}}(D_i, D_j), 0) \\ & + w_2 \cdot \max(\|D_i - D_j\| - 6, 0)^2 \\ & + w_3 \cdot \max(r_{ij} - N, 0)^2,\end{aligned} \tag{32}$$

where $\mathbf{1}_{\text{face}}(D_i, D_j)$ indicates whether the corresponding devices are placed face-to-face, and r_{ij} represents the ring loop of the corresponding devices. At each iteration, the algorithm evaluates the objective $\mathcal{F}(\mathcal{S})$ and constraint functions $\text{C}_{\text{viol}}(\mathcal{S})$ along with their gradients. For the i-th device located at coordinates (x_i, y_i), the gradients of the penalty function with respect to x_i are approximated as:

$$\nabla_{x_i}\text{C}_{\text{viol}} = \frac{\text{C}_{\text{viol}}\big(D_i^{(x_i+\delta)}, \{D_j\}_{j\neq i}\big) - \text{C}_{\text{viol}}\big(D_i^{(x_i-\delta)}, \{D_j\}_{j\neq i}\big)}{2\delta} \tag{33}$$

The placement $\mathcal{S}$ is updated accordingly until the gradients satisfy the stopping condition:

$$|\nabla\mathcal{F}| < \epsilon \quad \text{and} \quad |\nabla\text{C}_{\text{viol}}| < \epsilon, \tag{34}$$

indicating convergence to a locally optimal solution. The complete update process is summarized in Algorithm 2.

D. Legalization

The legalization algorithm resolves hard constraint violations from global placement to ensure design compliance. We introduce a fine-tuning strategy using SA with low temperature to explores discrete rotation angles via stochastic jumps, and a further gradient-based optimization to constrain the randomness of SA results. The details are shown in Algorithm 3.

The SA algorithm starts with an initial temperature T_0 and a cooling factor α to gradually reduce randomness. Since this stage corresponds to legalization, where fine-tuning is required, T_0 is set to a relatively small value of 10. In each iteration, a device and candidate angle are randomly selected to generate a new placement $\mathcal{S}$ with angle Θ. The objective function is evaluated, with worse solutions accepted probabilistically to avoid local optima:

$$p = \begin{cases} 1, & f' - f^* < 0 \\ e^{-\frac{f'-f^*}{T}}, & f' - f^* > 0 \end{cases} \tag{35}$$

where T controls the probability of accepting suboptimal solutions. After generating a new placement, a legality check is performed. If valid, the gradient optimization is applied to iteratively update device positions and angles by minimizing the objective function.

Combining SA with the gradient optimization allows the algorithm to flexibly jump in discrete angle space and avoid local optima, while fine-tuning continuous parameters to converge on high-quality solutions. Ultimately, the iterations yield the optimized layout $\mathcal{S}^*$ and angle combination Θ^*.

Algorithm 3 PCB Placement Legalization Process

Require: Global placement $\mathcal{S}_0$, angle set Θ, initial temperature T_0, cooling coefficient α, iteration limit N, objective $\mathcal{F}(\mathcal{S}, \Theta)$
Ensure: Optimized placement $\mathcal{S}^*$, optimized angles Θ^*

```
Initialize: Θ* = 0, S* = S_0, f* = F(S*, Θ*), T = T_0
for t = 1 : N do
    Randomly select a subset I ⊆ {1, ..., |D|}
    For each i ∈ I, sample θ'_i ~ U(Θ) and displacement Δp_i ~ U(P)
    Update Θ'_i ← θ'_i, p'_i ← p_i + Δp_i, construct S'
    if S' is legal then
        Gradient optimization: S_opt ← arg min_D F(S')
        Compute f' = F(S_opt, Θ'), Δf = f' − f*
        if Δf < 0 then
            Accept: S* ← S_opt, Θ* ← Θ', f* ← f'
        else
            Accept with prob. p = exp(−Δf/T) if u ~ U(0,1) ≤ p
        end if
    end if
    T ← αT
end for
return Optimized placement S*, optimized angles Θ*
```

V. Experiments

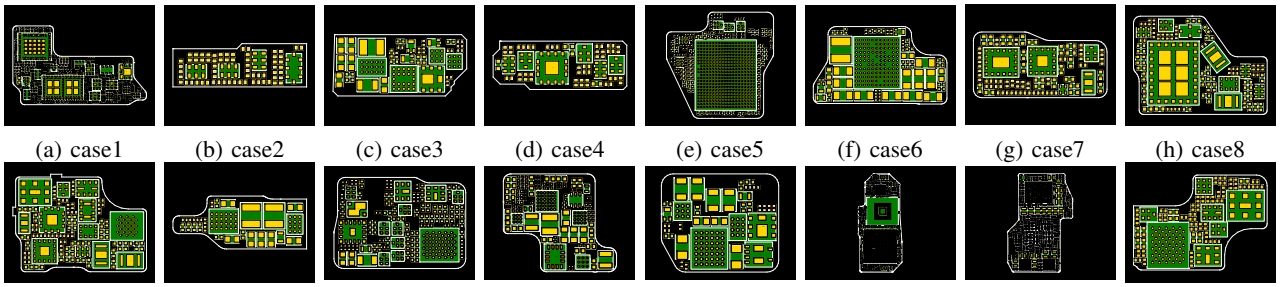
A. Experiment Setup

The computational setup for our experiments includes an Intel Core i9-14900K CPU with 24 cores and 128 GB of system RAM. The software environment is based on Python 3.10 (64-bit) running on a Linux operating system.

B. Dataset

1) Industrial Commercial Dataset

The industrial commercial dataset consists of 16 modern



(a) case1 (b) case2 (c) case3 (d) case4 (e) case5 (f) case6 (g) case7 (h) case8

(i) case9 (j) case10 (k) case11 (l) case12 (m) case13 (n) case14 (o) case15 (p) case16

Fig. 7. Visualization of the Modern Commercial Dataset. The PCB shapes are highly irregular, the layouts are extremely dense, and the device sizes vary considerably.

irregular PCB cases, as shown in Fig. 7. All of these cases are provided by Huawei Device Co., Ltd. And these cases all have corresponding manually created gold-standard layouts. These cases all have irregular boundary, extremely limited space for placement with high demand of surface-layer routability, and strict constraints of industrial PCBs. The routability of surface-layer traces is highly related to the quality of PCB layouts, which is an key metric for the evaluation of PCB placement results.

2) Open-Sourced Dataset

Besides, we also evaluate ISPCBPlace on the open-source PCBBenchmarks dataset [30], which provides 11 real PCB designs in KiCad format [31] along with corresponding manually routed layouts for benchmarking automated PCB layout tools. This dataset offers a standardized set of test cases to compare both placement and routing performance against manual designs.

C. Methods for Comparison

To comprehensively evaluate our proposed method, we compare it against two representative approaches: SA-PCB from the OpenROAD Project [5] and a state-of-the-art RL-based PCB placement method (RL-PCB) [6]. Additionally, we include manual placement as a baseline to assess the performance gap between automated and human-designed solutions. For open-source benchmarks, we also compare with common PCB layout tools including KiCad [31] and EasyEDA [32].

In the industrial PCB dataset, manual placements serve as the **gold standard** for evaluating layout quality. Crafted by expert engineers through extensive design cycles invoking surface-layer routing tools to obtain routability feedback, they represent near-optimal solutions that meet almost all constraints, including the most challenging surface-layer routing constraints, while pushing HPWL to its practical limit. The closer an automated method approaches this human benchmark, the more effective it is in minimizing wirelength and satisfying complex constraints.

D. Evaluation Metrics

1) Industrial Commercial Dataset

We evaluate the commercial PCB placement results using an evaluation program provided by Huawei Device Co., Ltd. This program reports multiple placement quality metrics, including Routability of Surface-Layer Traces (RSLT), HPWL, and Constraint Satisfaction Rate (CSR). The manual placement results are regarded as golden baselines for comparison. Our analysis focuses on the following key metrics:

- HPWL: A critical metric in PCB placement, reflecting the overall signal path length.
- CSR: A metric for evaluating PCB layout quality, defined as the percentage of PCB layouts that fully satisfy all design constraints.
- Time: Evaluates the computational cost of each method, reflecting its practicality in industrial design workflows.

2) Open-Source Dataset

For the open-source PCBBenchmarks dataset [30], we evaluate routing performance by applying automatic routing with the commercial PCB design tool Altium Designer [33]. The following metrics are considered:

- HPWL: The half-perimeter wirelength, widely used as a proxy for routability and a standard indicator in placement evaluation.
- WL: The wirelength obtained after detailed routing with Altium Designer [33], reflecting the practical routing quality of the placement results.

E. Experimental results

1) Comparison of HPWL and CSR under industrial commercial dataset

Table III summarizes the performance of different placement methods on 16 industrial commercial benchmarks. Based on these results, we draw the following observations:

- **HPWL Analysis.** HPWL is a key metric for layout quality. Manual layouts achieve short HPWL but are time-consuming, and any results exceeding 110% of manual HPWL are considered low quality. SA-PCB produces the highest HPWL (1.33× manual on average), while RL improves to 1.19× but still lags behind human performance. In contrast, ISPCBPlace reduces HPWL by 46% over SA-PCB and 39.2% over RL-PCB, approaching manual quality with full automation. These results of ISPCBPlace are attributed to the consideration of HPWL gradients and the efficient directional updates provided by gradient-based optimization.
- **Constraint Satisfaction Analysis.** The CSR is crucial for evaluation of PCB quality. Manual placement consistently achieves 100% CSR, but it achieves a higher HPWL

TABLE III
EXPERIMENTAL RESULTS COMPARISON ON INDUSTRIAL COMMERCIAL DATASET

Case	Manual Design (⋆Golden)			SA-PCB [5]			RL-PCB [6]			ISPCBPlace (ours)		
	HPWL(mm)↓	CSR(%)↑	Time(h)↓	HPWL(mm)↓	CSR(%)↑	Time(h)↓	HPWL(mm)↓	CSR(%)↑	Time(h)↓	HPWL(mm)↓	CSR(%)↑	Time(h)↓
case1	13346.25	100	≥12	25389.71	64	24.5	21506.95	70	15.7	**12535.52**	**86**	12.7
case2	858.88	100	≥1	1758.96	75	2.1	1659.27	85	1.7	**785.45**	**100**	**0.25**
case3	2092.49	100	≥1.5	3284.39	70	2.71	2956.87	80	1.92	**1830.99**	**96.7**	**0.57**
case4	804.26	100	≥1	1784.91	80	2.26	1489.72	85	1.76	**713.84**	**100**	**0.56**
case5	7273.15	100	≥6	9952.84	65	15.6	8259.14	75	12.2	**6467.65**	**90**	6.51
case6	5274.16	100	≥4	8822.17	65	7.1	7168.48	73.3	5.6	**4540.93**	**90**	**0.84**
case7	1811.88	100	≥3	2527.98	72.5	5.4	2708.17	75	4.1	**1492.69**	**92.5**	**1.95**
case8	1781.53	100	≥3	2984.62	81	6.24	2748.92	84	5.47	1787.81	**98**	1.95
case9	2196.28	100	≥2	3158.14	76	5.7	2857.47	79	4.6	**1872.52**	**96**	2.31
case10	1395.22	100	≥1	1928.53	72.5	2.25	1795.81	77.5	1.84	**1323.51**	**90**	**0.18**
case11	6087.75	100	≥6	8961.71	65	9.5	7794.32	70	6.7	**5396.68**	**92**	**4.3**
case12	5695.70	100	≥7.5	6894.96	75	12.1	6259.53	85	9.7	**3605.07**	**94**	**5.2**
case13	2222.63	100	≥1.5	3284.39	70	2.71	2956.87	80	1.92	**1569.79**	**95**	**0.57**
case14	70800.72	100	≥20	84726.91	60.0	36.8	77840.69	66.6	29.4	**47761.33**	**86.7**	25.8
case15	36406.63	100	≥15	44894.06	65	25.6	39973.63	75	23.2	**21770.73**	**85**	20.5
case16	3682.2	100	≥2	4789.21	75	3.1	4691.03	75	3.9	**3929.12**	**95**	**1.1**
ratio	1.0	1.0	1.0	1.330	0.707	1.892	1.191	0.772	1.499	0.725	0.929	0.986

TABLE IV
EXPERIMENTAL RESULTS COMPARISON ON OPEN-SOURCE DATASET [30]

Case	Manual Design		SA-PCB [5]		RL-PCB [6]		KiCad [31]		EasyEDA [32]		ISPCBPlace (ours)	
	HPWL(mm)↓	WL(mm)↓	HPWL(mm)↓	WL(mm)↓	HPWL(mm)↓	WL(mm)↓	HPWL(mm)↓	WL(mm)↓	HPWL(mm)↓	WL(mm)↓	HPWL(mm)↓	WL(mm)↓
bm1	4056.42	5034.78	3184.25	4486.42	3506.92	4693.02	3537.25	4535.91	5164.58	7632.27	**2399.74**	**4105.63**
bm2	228.21	**395.83**	371.68	612.34	378.14	632.74	-	-	-	-	**153.06**	430.16
bm3	771.20	1300.63	644.38	1413.43	719.28	1597.82	-	-	919.98	1814.81	**537.94**	**1198.89**
bm4	717.98	**1043.25**	571.03	979.43	623.81	1421.50	798.18	1300.99	946.55	1739.21	**469.74**	1096.35
bm5	342.66	681.61	265.35	572.04	309.21	670.21	417.72	819.54	-	-	**288.73**	**513.38**
bm6	665.78	778.84	595.97	863.14	601.43	809.76	-	-	960.32	1532.99	**591.72**	**753.19**
bm7	60.04	128.27	49.24	119.97	56.07	167.98	80.85	165.23	93.93	196.52	**43.07**	**91.35**
bm8	881.3	1483.38	976.17	1322.73	908.95	1290.85	1221.05	1661.40	-	-	**829.43**	**1046.28**
bm9	2772.01	3721.20	1809.45	2857.87	1946.32	3476.01	2003.01	3157.17	2399.02	4006.10	**879.2**	**2055.42**
bm10	1034.39	1743.43	791.05	1499.42	908.72	1679.04	941.56	1659.62	1200.12	2445.76	**536.62**	**1259.15**
bm11	797.48	1120.59	704.18	1414.57	765.09	1398.03	-	-	-	-	**612.2**	**1059.47**
ratio	1.0	1.0	0.808	0.926	0.870	1.023	1.147	1.199	1.489	1.746	**0.596**	**0.781**

Note: "-" indicates that the method failed to produce valid placement result for the corresponding benchmark.

compared to ISPCBPlace. SA-PCB struggles with complex PCB constraints, reaching only 70.7% on average, which highlights its difficulty in handling irregular PCB constraints. RL-based placement slightly improves CSR to 77.2%, yet still falls short of practical requirements. ISPCBPlace significantly outperforms both methods, attaining an average CSR of 92.9%, which demonstrates its effectiveness in adhering to PCB placement constraints. The high level of constraint satisfaction is attributed to ISPCBPlace's explicit constraint modeling and the efficient search capability of its legalization process.

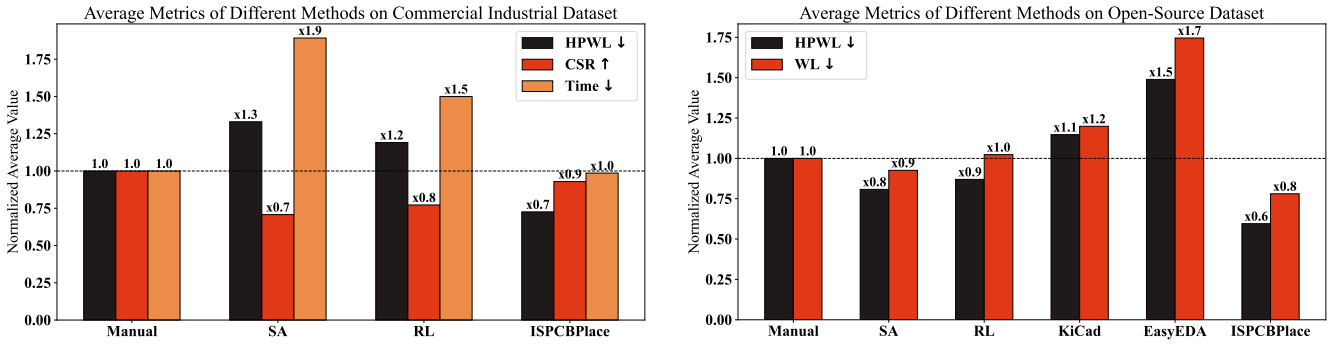
- **Runtime Efficiency and Scalability.** While manual placement achieves the highest level of constraint satisfaction, it requires significantly more design time and is thus impractical for large-scale layouts. Among automated methods, SA-PCB incurs the longest runtime (1.91× that of ISPCBPlace) due to its iterative annealing process and random search, whereas RL-based placement is moderately faster but still takes 1.52× the runtime of ISPCBPlace. In contrast, ISPCBPlace consistently outperforms both methods across all test cases, with the advantage becoming more pronounced for large-scale designs. Depending on design complexity, the runtime of ISPCBPlace ranges from as little as 0.18 hours (**case**10) to 25.8 hours (**case**14). Notably, it completes moderately complex layouts such as **case**5 within 6.51 hours, and even the complex case (**case**1) within 13 hours. The majority of small-scale to medium-scale designs are finished within 2 hours, demonstrating both the scalability and computational efficiency of ISPCBPlace in irregular PCB placement tasks. This runtime efficiency is attributed to ISPCBPlace's gradient-based optimization, which effectively handles complex constraints while avoiding excessive exploration overhead.

Fig. 8 (a) summarizes the average performance of different placement methods under industrial commercial dataset. As shown in the left panel, manual placement serves as the baseline with normalized values of 1.0 across all metrics. SA-PCB suffers from excessive runtime (1.9×) due to its iterative annealing process, while also exhibiting higher HPWL (1.3×) and lower CSR (0.7×). RL-based placement offers improvements, reducing HPWL to 1.2× and CSR to 0.8×, but still incurs a runtime of 1.5×. In contrast, ISPCBPlace achieves the best trade-off among automated approaches, with HPWL reduced to 0.7×, CSR close to the baseline (0.9×), and runtime efficiency comparable to manual placement (1.0×).

2) Comparison of HPWL and Post-Routing Wirelength under open-sourced dataset

Table IV summarizes the performance of different placement methods on 11 open-source benchmarks, where **WL** denotes the post-routing wirelength obtained using Altium Designer [33]. Based on these results, we make the following observations:

- **HPWL Analysis.** ISPCBPlace achieves a normalized HPWL of only $0.596\times$ that of manual placement, corresponding to an average reduction of $\mathbf{40.4}\%$. Compared to prior automated approaches, it further reduces HPWL by 26.3% relative to SA-PCB ($0.808\times$) and 31.5%



(a) Comparison on industrial commercial dataset.

(b) Comparison on open-source dataset.

Fig. 8. Comparison of normalized average metrics across different placement methods. (a) Evaluation of HPWL, CSR, and runtime efficiency, normalized to manual placement. (b) Evaluation of HPWL and WL across both academic and commercial EDA tools, with manual placement as the baseline.

relative to RL ($0.870\times$). In contrast, industrial tools such as KiCad and EasyEDA yield higher HPWL ratios (1.147 and 1.489, respectively), demonstrating the clear advantage of ISPCBPlace over both academic and commercial baselines. On individual benchmarks, substantial improvements are observed: for instance, in *bm1*, HPWL is reduced by 40.8% (from 4056.4mm to 2399.7mm); in *bm3*, by 30.2% (from 771.2mm to 537.9mm); and in *bm9*, by 68.3% (from 2772.0mm to 879.2mm). These results highlight the effectiveness of ISPCBPlace in consistently minimizing HPWL across diverse PCB designs.

- **WL Analysis.** In terms of Post-routing wirelength (WL), ISPCBPlace achieves an average normalized ratio of $0.781\times$ compared with manual placement, corresponding to a mean reduction of $\mathbf{21.9}\%$. Relative to prior methods, it achieves lower WL than SA-PCB ($0.926\times$) and RL ($0.961\times$), while significantly outperforming industrial tools KiCad ($1.199\times$) and EasyEDA ($1.746\times$). Per-benchmark results further confirm its scalability: in *bm1*, WL is reduced by 18.4%; in *bm3*, by 7.7%; and in *bm9*, by 44.8%. These improvements demonstrate that ISPCBPlace not only reduces HPWL but also effectively shortens post-routing wirelength, which is crucial for improving routability and signal integrity in large-scale PCB layouts.

Fig. 8 (b) further compares HPWL and overall WL across both academic and industrial EDA tools. Open-source tools such as KiCad and EasyEDA yield significantly higher HPWL and WL, particularly EasyEDA (1.5× HPWL and 1.7× WL). SA-PCB and RL demonstrate better performance but still remain above the manual baseline. Notably, ISPCBPlace achieves the lowest normalized values (0.6× HPWL and 0.8× WL), clearly outperforming both academic baselines and commercial design tools.

F. PCB Constraints Analysis for ISPCBPlace

Table V provides a more detailed display of ISPCBPlace's satisfaction with various complex constraints on irregular PCB cases. ISPCBPlace is evaluated across 16 representative industrial cases, covering a wide range of layout complexities. Specifically, ISPCBPlace achieves a constraint satisfaction rate between 86% and 100%, with an average of 92.9%, highlighting its consistent reliability across diverse placement scenarios.

1) Hard Constraints

The hard constraints refer to the mandatory conditions that must be satisfied, including the boundary constraint, spacing constraint, total layout area limited to no more than 120% of the manual design, and HPWL restricted to less than 110% of the manual design. The satisfaction of these hard constraints serves as a fundamental criterion for ensuring high-quality PCB placement and manufacturability. ISPCBPlace maintains 100% compliance with hard constraints across all test cases. This ensures that all placed components respect physical board outlines and required spacing margins for manufacturability. These results are attributed to ISPCBPlace's effective handling of hard constraints, demonstrating the practicality of ISPCB-Placein modern irregular PCB placement and highlight the high-quality performance of its gradient optimization.

2) Soft Constraints

The soft constraints refer to the conditions that are desirable to satisfy but not strictly mandatory, including RSLT and module-associated proximity constraints, such as Isolated Devices, Surface-Layer Fanout, and Module Boundary Clear. These constraints do not affect the legality of the layout, but satisfying them generally leads to improved layout quality and performance. RSLT includes routability of surface-layer traces of critical devices, non-critical devices and other devices. While ISPCBPlace satisfies critical RSLT in most cases, a small percentage of violations remain in isolated scenarios. However, non-critical and other RSTL are largely satisfied, highlighting ISPCBPlace's robustness in handling PCB-specific constraints. As for module-associated proximity constraints, ISPCBPlace satisfies nearly all of them, owing to its effective clustering and high-granularity optimization.

3) Overall Constraint Satisfaction

The constraint satisfaction rate ranges from 86% to 100% across all cases, with an average of over 92.9%, proving the effectiveness of ISPCBPlace in managing irregular PCB placement challenges. The constraint satisfaction rate ranges from 86% (**case**1) to 100% (**cases** 2, 4). Most test cases achieve $\geq 90\%$ satisfaction, validating ISPCBPlace's high de-

TABLE V
DRC COMPLIANCE OF ISPCBPLACE UNDER INDUSTRIAL PCB CASES

Indicator	case1	case2	case3	case4	case5	case6	case7	case8	case9	case10	case11	case12	case13	case14	case15	case16
Number of components	220	41	35	35	97	57	63	68	45	25	97	77	33	261	277	59
Runtime(h)	12.7731	0.2553	0.5703	0.5670	6.5169	0.8497	1.9539	1.9536	2.3131	0.1820	4.3	5.2	0.57	25.8	20.5	1.1
Device within Boundary	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
100% Device Spacing Met	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
Area $\leq$ 120% of Manual	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
HPWL $\leq$ 110% of Manual	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
Module Boundary Clear	pass	pass	pass	pass	pass	pass	pass	pass	pass	1/2	pass	pass	pass	pass	pass	pass
Surface-Layer Fanout	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
Isolated Devices	3/5	pass	pass	pass	2/3	pass	3/4	pass	4/5	1/2	4/5	4/5	pass	2/3	2/4	pass
RSTL of Critical Devices	3/5	pass	pass	pass	pass	pass	pass	4/5	4/5	pass	pass	pass	3/4	2/3	3/4	pass
RSTL of Other Devices	4/5	pass	2/3	pass	pass	pass	pass	pass	pass	pass	4/5	4/5	3/4	2/3	3/4	1/2
RSTL of Non-Critical Devices	3/5	pass	pass	pass	1/3	0/1	2/4	pass	pass	pass	3/5	4/5	pass	2/3	2/4	1/2
Constraint satisfaction rate	86%	100%	96.7%	100%	90%	90%	92.5%	98%	96%	90%	92%	94%	95%	86.7%	85%	95%

gree of constraint compliance. The slightly lower satisfaction in complex layouts such as case1 (with 220 components and 5 constraint types partially unmet) reflects the inherent trade-offs in ultra-dense placements but still surpasses other benchmarks.

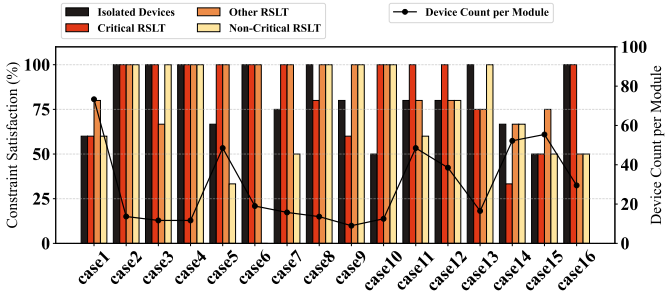


Fig. 9. The relationship between RSLT and the maximum number of devices within each module.

4) Analysis of RSLT vs. Device Count per module

The surface-layer routing constraint represents a major challenge in automated PCB placement. Fig. 9 illustrates the relationship between RSLT and the maximum number of devices within each module. We observe a clear trend:

- **Low device density.** Cases with relatively few devices per module (e.g., case2, case6, case7, case8) are associated with near-perfect satisfaction, often achieving 100%. This suggests that a smaller number of devices allows the optimizer more flexibility to distribute them in ways that better meet spatial and directional constraints.
- **High device density.** Modules with a large number of devices generally exhibit *lower* constraint satisfaction across all four categories. For example, case1 has 80 components in the most crowded module but shows constraint satisfaction below 80% in all categories.
- **Complex geometric and topological conditions.** Certain cases deviate from the density-satisfaction trend. Case5, although having slightly more devices, still suffers from low satisfaction, indicating that complexity is not only related to quantity but also to the *geometry* and *topology* of module regions, such as long narrow zones and presence of dense surface-layer trace constraints.

Even under the stringent surface-layer routing constraints, ISPCBPlace demonstrates superior robustness, outperforming other automated PCB placement methods and producing layouts most comparable to manual designs.

TABLE VI
EXPERIMENTAL RESULTS COMPARISON WITH DREAMPLACE [15]

Case	Dreamplace [15]			ISPCBPlace (ours)		
	gHPWL(mm)↓	gOverlap(%)↓	lHPWL(mm)↓	gHPWL(mm)↓	gOverlap(%)↓	lHPWL(mm)↓
case1	13791.83	2.2	14338.42	**12532.52**	**0**	**12532.52**
case2	936.44	10.0	977.28	**784.98**	**2.5**	**785.45**
case3	1955.44	18.2	2017.03	**1824.81**	**8.6**	**1830.99**
case4	1073.62	14.4	1101.64	**621.82**	**9.9**	**713.84**
case5	6219.74	6.3	6592.19	**5860.1**	**2.1**	**6467.65**
case6	5174.81	15.6	5401.24	**4327.02**	**5.2**	**4540.93**
case7	2288.56	3.6	2292.15	**1337.95**	**1.2**	**1492.69**
case8	1823.52	22.3	2051.05	**1734.83**	**14.8**	**1787.81**
case9	2437.22	25.3	2546.85	**2021.18**	**12.6**	**1872.52**
case10	1846.36	11.1	1846.36	**1265.39**	**3.7**	**1323.51**
case11	5827.02	3.1	6062.94	**4962.01**	3.1	**5396.48**
case12	4312.53	6.5	4312.53	**4446.72**	6.5	**3605.07**
case13	1838.33	6.1	1963.14	**1615.81**	**3.1**	**1569.79**
case14	52031.25	2.7	53702.14	**50983.25**	**1.9**	**47761.33**
case15	25897.92	5.7	26781.14	**24580.53**	**5.4**	**21770.73**
case16	4827.71	**5.6**	4975.03	**4603.91**	6.8	**3929.12**
ratio	1.0	1.0	1.0	0.93	0.55	0.95

G. Comparison with IC Gradient-based Placement Method

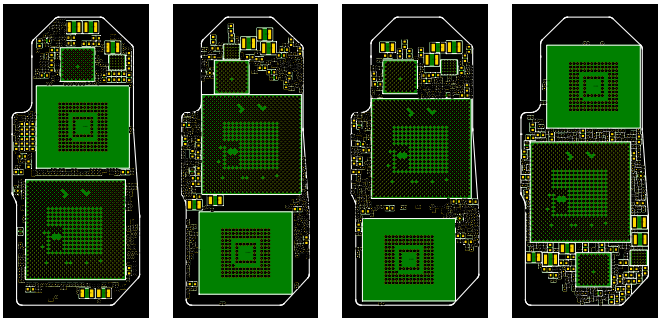
We conduct an additional experiment by comparing ISPCBPlace with the commonly used smooth HPWL approximation adopted in DreamPlace [15]. Specifically, DreamPlace replaces the non-differentiable max/min operators in HPWL with a weighted-average (WA) formulation to enable gradient-based optimization:

$$\mathrm{WA}_e = \frac{\sum_{i\in e} x_i e^{\frac{x_i}{\gamma}}}{\sum_{i\in e} e^{\frac{x_i}{\gamma}}} - \frac{\sum_{i\in e} x_i e^{-\frac{x_i}{\gamma}}}{\sum_{i\in e} e^{-\frac{x_i}{\gamma}}}, \tag{36}$$

where γ represents the smoothness and accuracy of the approximation to HPWL. Following common practice in Dreamplace [15], we set $\gamma = 4$ in all experiments. While this approximation provides continuous gradients, it inevitably introduces geometric bias in wirelength estimation, particularly when pin distributions are highly clustered. In contrast, ISPCBPlace evaluates HPWL directly based on exact pin extrema without resorting to smoothing, thereby enabling high-granularity wirelength optimization. Such accuracy is especially critical for PCB layouts, where routing resources are limited and placement decisions are highly sensitive to local geometric variations.

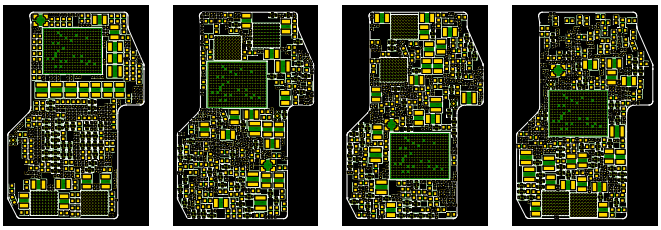
As reported in Table VI, **gHPWL** denotes the HPWL evaluated on the global placement with overlaps allowed, **lHPWL** represents the HPWL after legalization, and **gOverlap** measures the percentage of overlapping components in the global placement. Across all industrial PCB benchmarks, ISPCBPlace consistently achieves lower gHPWL and lHPWL than DreamPlace [15]. The improvement is particularly pronounced in complex cases with dense pin clusters, demonstrating that

accurate and fine-grained HPWL evaluation plays a critical role in guiding optimization toward high-quality PCB layouts. In addition, fine-grained overlap-constraint modeling enables more effective optimization of PCB placement by providing more precise guidance in irregular and densely populated design scenarios.



(a) Manual (b) SA-PCB [5] (c) RL-PCB [6] (d) ISPCBPlace

Fig. 10. Visualization of industrial case14.



(a) Manual (b) SA-PCB [5] (c) RL-PCB [6] (d) ISPCBPlace

Fig. 11. Visualization of industrial case15.

H. Case Studies

Fig. 10 and Fig. 11 presents the visualization results of industrial PCB placement cases provided by Huawei Device Co., Ltd. These cases involve highly constrained and irregularly shaped PCB layouts, posing significant challenges for automated placement.

The manual designs are treated as golden runs, as they are produced by expert PCB designers through an iterative placement–routing–verification process that simultaneously achieves near-optimal HPWL and full constraint satisfaction. In contrast, existing automated methods including SA-PCB [5] and RL-PCB [6] struggle to balance wirelength optimization and constraint compliance under irregular boundaries, limited placement space, and large component size variations.

ISPCBPlace aggressively optimizes HPWL while maintaining a high level of constraint satisfaction by formulating objectives and constraints within a unified gradient-based framework. It can be observed that ISPCBPlace produces layouts with overall organization comparable to the manual designs, including similar module-level arrangements related to surface-layer routing. This behavior can be attributed to the proposed module clustering strategy together with the accurate HPWL optimization. Although the routability of surface-layer traces remains difficult to model explicitly, ISPCBPlace consistently achieves significantly lower HPWL than prior automated approaches. Benefiting from both high-quality wirelength optimization and a high degree of constraint satisfaction, ISPCBPlace can serve as a practical reference solution for expert designers, effectively reducing manual effort in PCB placement.

VI. Conclusion

PCB placement remains largely manual due to challenges like irregular shapes, high density, component size variations, and routing constraints—hindering full automation. We propose ISPCBPlace, a gradient-based method for industrial PCB placement. We formulate global placement as a gradient optimization problem using three key gradients, enabling constraint-aware optimization. We also propose module-aware clustering for improved surface routability and a simulated annealing strategy for orientation legalization. Experiments on real-world PCBs show that ISPCBPlace outperforms state-of-the-art methods and manual designs in wirelength while satisfying all hard industrial constraints. In future work, we will extend ISPCBPlace to larger-scale PCBs such as mainboards and further accelerate the placement optimization process, and release a new open-source benchmark dataset inspired by real industrial cases.

Acknowledgments

No any generative AI was used in preparation of this paper.

References

[1] S. Jain and H. C. Gea, "Pcb layout design using a genetic algorithm," *Journal of Electronic Packaging*, vol. 118, pp. 11–15, 1995. [Online]. Available: https://api.semanticscholar.org/CorpusID:110688179

[2] F. S. Ismail, R. Yusof, and M. Khalid, "Optimization of electronics component placement design on pcb using self organizing genetic algorithm (soga)," *J. Intell. Manuf.*, vol. 23, no. 3, p. 883–895, Jun. 2012. [Online]. Available: https://doi.org/10.1007/s10845-010-0444-x

[3] T. Badriyah, F. Setyorini, and N. Yuliawan, "The implementation of genetic algorithm and routing lee for pcb design optimization," in *2016 International Conference on Informatics and Computing (ICIC)*, 2016, pp. 148–153.

[4] A. Alexandridis, E. Paizis, E. Chondrodima, and M. Stogiannos, "A particle swarm optimization approach in printed circuit board thermal design," *Integr. Comput.-Aided Eng.*, vol. 24, no. 2, p. 143–155, Jan. 2017. [Online]. Available: https://doi.org/10.3233/ICA-160536

[5] T. O. Project, "Sa-pcb: Annealing-based pcb placement tool," https://github.com/The-OpenROAD-Project/SA-PCB.

[6] L. Vassallo and J. Bajada, "Learning circuit placement techniques through reinforcement learning with adaptive rewards," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024, pp. 1–6.

[7] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nazi, J. Pak, A. Tong, K. Srinivasa, W. Hang, E. Tuncer, Q. V. Le, J. Laudon, R. Ho, R. Carpenter, and J. Dean, "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, 2021.

[8] Y. Lai, Y. Mu, and P. Luo, "Maskplace: Fast chip placement via reinforced visual representation learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 019–24 030, 2022.

[9] Y. Lai, J. Liu, Z. Tang, B. Wang, J. Hao, and P. Luo, "Chipformer: Transferable chip placement via offline decision transformer," in *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 18 346–18 364.

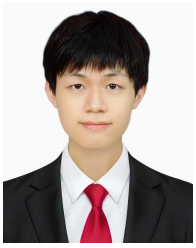
[10] D. Guerra, A. Canelas, R. Póvoa, N. Horta, N. Loureno, and R. Martins, "Artificial neural networks as an alternative for automatic analog ic placement," *IEEE*, 2019.

[11] A. Gusmão, F. Passos, R. Póvoa, N. Horta, N. Lourenço, and R. Martins, "Semi-supervised artificial neural networks towards analog ic placement recommender," in *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2020, pp. 1–5.
[12] A. Gusmo, R. Póvoa, N. Horta, N. Loureno, and R. Martins, "Deep-placer: A custom integrated opamp placement tool using deep models," *Appl. Soft Comput.*, vol. 115, p. 108188, 2021.
[13] J. Lu, H. Zhuang, P. Chen, H. Chang, C.-C. Chang, Y.-C. Wong, L. Sha, D. Huang, Y. Luo, C.-C. Teng, and C.-K. Cheng, "eplace-ms: Electrostatics-based placement for mixed-size circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 5, pp. 685–698, 2015.
[14] C.-K. Cheng, A. B. Kahng, I. Kang, and L. Wang, "Replace: Advancing solution quality and routability validation in global placement," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 9, pp. 1717–1730, 2019.
[15] Y. Lin, Z. Jiang, J. Gu, W. Li, S. Dhar, H. Ren, B. Khailany, and D. Z. Pan, "Dreamplace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 4, pp. 748–761, 2021.
[16] H. C. Ou, K. H. Tseng, J. Y. Liu, I. P. Wu, and Y. W. Chang, "Layout-dependent effects-aware analytical analog placement," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 8, pp. 1243–1254, 2016.
[17] M. Liu, K. Zhu, X. Tang, B. Xu, and D. Z. Pan, "Closing the design loop: Bayesian optimization assisted hierarchical analog layout synthesis," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020.
[18] Y. Lin, Y. Li, D. Fang, M. Madhusudan, S. S. Sapatnekar, R. Harjani, and J. Hu, "Are analytical techniques worthwhile for analog ic placement?" in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2022, pp. 154–159.
[19] H. Chen, M. Liu, B. Xu, K. Zhu, X. Tang, S. Li, Y. Lin, N. Sun, and D. Z. Pan, "Magical: An open- source fully automated analog ic layout system from netlist to gdsii," *IEEE Design & Test*, vol. 38, no. 2, pp. 19–26, 2021.
[20] H. Murata, K. Fujiyoshi, S. Nakatake, and Y. Kajitani, "Vlsi module placement based on rectangle-packing by the sequence-pair," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 15, no. 12, pp. 1518–1524, 1996.
[21] J. Cohoon and W. Paris, "Genetic placement," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 6, no. 6, pp. 956–964, 1987.
[22] B. Goplen and S. Sapatnekar, "Placement of thermal vias in 3-d ics using various thermal objectives," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 4, pp. 692–709, 2006.
[23] H. Liang, S. Sinha, and W. Zhang, "Parallelizing hardware tasks on multicontext fpga with efficient placement and scheduling algorithms," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 2, pp. 350–363, 2018.
[24] R. Martins, N. Lourenço, and N. Horta, "Laygen ii—automatic layout generation of analog integrated circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 32, no. 11, pp. 1641–1654, 2013.
[25] J.-E. Chen, P.-W. Luo, and C.-L. Wey, "Placement optimization for yield improvement of switched-capacitor analog integrated circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 29, no. 2, pp. 313–318, 2010.
[26] C.-K. Cheng, C.-T. Ho, and C. Holtz, "Net separation-oriented printed circuit board placement via margin maximization," in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2022, pp. 288–293.
[27] Z. Zhu, Y. Li, M. Su, S. Zhang, H. Su, Y. Xiao, H. He, J. Chen, and Y.-W. Chang, "Subgraph matching-based reference placement for printed circuit board designs," *The Journal of Supercomputing*, vol. 80, no. 16, pp. 24 324–24 357, nov 2024. [Online]. Available: https://doi.org/10.1007/s11227-024-06338-9
[28] C. Chen, H. Lin, M. Su, H. He, J. Chen, and Z. Zhu, "Subgraph matching with diversity handling and its applications to pcb placement," in *2024 2nd International Symposium of Electronics Design Automation (ISEDA)*, 2024, pp. 468–473.
[29] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, Mar. 2020. [Online]. Available: https://doi.org/10.1038/s41592-019-0686-2
[30] Github, "Pcbbenchmarks: Kicad pcb benchmarks for automated layout," https://github.com/aspdac-submission-pcb-layout/PCBBenchmarks.
[31] KiCad, "Kicad electronic design automation suite," https://www.kicad.org/, 2025, version 9.0.2.
[32] EasyEDA, "Easyeda pro edition," https://easyeda.com/, 2025, version 2.2.40.2.
[33] Altium, "Altium designer," https://www.altium.com/, 2025, version 25.7.1.

Lei Cai received the B.S. degree from Hubei University of Technology, Wuhan, China, in 2023. He is currently a graduate student pursuing the M.S. degree with the School of Computer Science and Artificial Intelligence, Wuhan University of Technology, Wuhan, China. He was previously involved in a Huawei project, where he worked on developing automated PCB placement and routing tools. His research interests include electronic design automation, and artificial intelligence.

Jixin Zhang received the Ph.D. degree from Hunan University, Hunan, China. He was a visiting scholar with The Chinese University of Hong Kong. He is currently an Associate Professor with the School of Computer Science, Hubei University of Technology, Wuhan, China. He previously led the development of automated PCB placement and routing tools for a Huawei project and was awarded the Huawei "Spark Award". His research interests include electronic design automation, and artificial intelligence.

Ke Cheng received the B.S. degree from Hubei University of Technology, Wuhan, China, in 2023. He is currently a graduate student pursuing the M.S. degree with the School of Computer Science, Hubei University of Technology, Wuhan, China. He was previously involved in a Huawei project, where he worked on developing automated PCB placement and routing tools. His research interests include electronic design automation, and artificial intelligence.



Haiyun Li received the B.S. degree from Huazhong University of Science and Technology in 2019, and the M.S. degree in Electronic Information from Wuhan University of Technology, Wuhan, China, in 2023. He is currently a Ph.D. student in Electronic Information at the Shenzhen International Graduate School, Tsinghua University. He was previously involved in a Huawei project, where he worked on developing automated PCB placement and routing tools. His research interests include artificial intelligence and electronic design automation.

Zhiwei Ye received the Ph.D. degree in the school of remote sensing and information engineering at Wuhan University, Wuhan, China. He serves as the Dean of the School of Computer Science, Hubei University of Technology, Wuhan, China. He also holds concurrent positions as the Executive Dean of the School of Big Data and Artificial Intelligence (Hubei Provincial Modern Industry College), Director of the Hubei Provincial Key Laboratory of Green Intelligent Computing Networks, and Director of the Hubei Provincial Engineering Research Center of Intelligent Manufacturing Technology and Applications. His research interests include artificial intelligence.

Mingyu Liu received the B.S. degree from Huazhong University of Science and Technology, Wuhan, China. He is a Senior Technical Expert at Huawei Co., Ltd. He is an Executive Committee Member of the CCF Technical Committee on Integrated Circuits. He has over 20 years of experience in product development and tool design. He was responsible for the planning and implementation of the terminal hardware toolchain at Huawei Co., Ltd. His research interests include electronic design automation, PCB automated testing tools.